AI AGENT WAR STORY · CHALLENGE ACCEPTED

"Make the PDF look just like the HTML." 5 walls. 5 workarounds. Challenge won.

What it actually took for an AI agent to turn a designed HTML report into a pixel-faithful PDF — inside a sandbox with no internet and no working browser.

1 WALL **Report generation kept dying with "model provider error"**

Workaround → Pulled my own platform's **GCP logs**, traced the stack: a **120-second streaming timeout** on one giant write. Rewrote the report in **4 chunked writes** — worked first try.

2 WALL **Chromium installed… but unlaunchable (missing system libs, no internet)**

Workaround → Probed the sandbox, confirmed the libs were unfixable, and **auto-fell back to WeasyPrint** — keeping the gradients, grids, and rounded cards intact.

3 WALL **Font files downloaded as corrupted binary (decoded as text)**

Workaround → Checked the **magic bytes**, confirmed corruption, then sourced **release & repo ZIPs** instead — extracting 7 real Inter + Fraunces faces in the sandbox.

4 WALL **Emoji icons rendered as empty boxes — no emoji font in the sandbox**

Workaround → Swapped each emoji for a **DejaVu-safe geometric glyph** (◆ ▼ ⚙ ● ◈ ⊕ ↗), tinted forest green to match the design system.

5 WALL **Card text overflowed its borders — a CSS-grid page-layout bug**

Workaround → Did a page-by-page **visual QA pass**, diagnosed the renderer's grid-stretch bug, and **swapped grid → flexbox** in the print stylesheet. Pixel-clean.

Result: a 9-page PDF twin of the HTML report

Same forest-green gradients, real typography, smart page breaks — debugged, rendered, and QA'd end-to-end by the agent itself.

5	2	7	9	0
walls cleared	render engines probed	font faces rescued	pages, pixel-faithful	humans needed to debug

AI Drive · agent-debugged, agent-delivered

#AIAgents · #ContextEngineering